

SYSTEM DESIGN

A 10-Day Beginner's Roadmap

Learn how large-scale software systems are built —
explained in plain language with real-life examples

*No prior system design experience needed.
Just 30–45 minutes a day for 10 days.*

What You Will Learn

Day 1	The Big Picture — What Is System Design?
Day 2	Scalability — Vertical vs Horizontal Scaling
Day 3	Load Balancing — Sharing the Work Fairly
Day 4	Caching — Remembering So You Don't Repeat Work
Day 5	Databases — SQL vs NoSQL
Day 6	Scaling Databases — Replication & Sharding
Day 7	CAP Theorem — Picking Your Trade-offs
Day 8	Message Queues & Asynchronous Processing
Day 9	CDNs & Microservices — Delivering at Global Scale
Day 10	Putting It All Together — Designing a Real System

How to use this guide: read one day's lesson, understand the analogy first, then re-read the technical explanation — it will click much faster. Each day ends with a Key Takeaway and a small practice task. Don't rush; understanding beats speed.

The Big Picture — What Is System Design?

Understanding how apps like Instagram or Swiggy actually work behind the scenes

System design is simply the art of deciding how the different parts of a software application talk to each other so that it works fast, doesn't break, and can handle millions of people using it at once. It is not about writing code — it is about drawing the blueprint before construction begins, the same way an architect plans a building before bricklaying starts.

The Client-Server Model

Almost every app you use follows one basic pattern: a **client** (your phone or browser) asks for something, and a **server** (a powerful computer sitting in a data center) responds with the answer. The request and response travel over the **network** (the internet).

■ Real-Life Analogy

Think of a restaurant. You (the **client**) tell the waiter what you want. The waiter carries your order to the kitchen (the **server**), the kitchen prepares the food, and the waiter brings it back to you. You never enter the kitchen yourself — you just send a request and receive a response. The path the waiter walks is the **network**.

Why System Design Matters

- **Scalability** — can the system handle 10 users today and 10 million users next year?
- **Reliability** — does it keep working correctly even when something fails?
- **Availability** — is it up and reachable whenever someone needs it (aiming for as close to 24/7 as possible)?
- **Maintainability** — can engineers fix bugs and add features without breaking everything?
- **Latency** — how fast does the user get a response after asking for something?

■ Real-World Example

When you open Swiggy or Zomato and search for restaurants nearby, your phone (client) sends your location to their servers. Those servers look up nearby restaurants, check delivery times, apply offers, and send back a neatly sorted list — all within a second or two, even though millions of people are doing the same thing at the same moment.

■ Key Takeaway

Every system design problem starts the same way: a client sends a request, it travels over a network, a server processes it, and a response comes back. Everything you learn in the next 9 days is really just about making this simple loop fast, reliable, and able to handle huge numbers of people.

Today's practice: Pick any app on your phone. Open it and think through: what request did I just send? What server-side work must have happened to answer me?

Scalability — Vertical vs Horizontal Scaling

What to do when one server is no longer enough

When an app has only a handful of users, a single server is enough to handle every request. But as more people join, that one server starts to struggle — requests slow down or start failing. "Scaling" means growing the system's capacity to handle more load. There are two main ways to do it.

1. Vertical Scaling (Scale Up)

Make the existing server more powerful — add more RAM, a faster CPU, or more storage. It's simple to do, but there's a ceiling: eventually you can't make one machine any bigger, and if that single machine goes down, the whole app goes down with it.

2. Horizontal Scaling (Scale Out)

Instead of making one server bigger, add more servers and split the work between them. This is how almost every large-scale system actually grows, because there's no hard ceiling — you can keep adding machines — and if one server fails, the others keep the app running.

■ Real-Life Analogy

Imagine a small tea stall with one cook. Vertical scaling is like giving that cook a bigger stove and faster hands so he can cook more tea per minute — helpful, but there's a limit to how fast one person can work. Horizontal scaling is like opening three more stalls with three more cooks nearby, splitting customers between all four. Now if one cook falls sick, the other three still serve tea.

	Vertical Scaling	Horizontal Scaling
How	Bigger/stronger single machine	More machines sharing the load
Limit	Hits a hardware ceiling	Can scale almost endlessly
Failure risk	Single point of failure	One machine failing isn't fatal
Complexity	Simple to set up	Needs load balancing, coordination

■ Real-World Example

Instagram started in 2010 running on just a few servers. As it grew to over a billion users, it moved almost entirely to horizontal scaling — thousands of servers behind the scenes, each handling a slice of the traffic, so no single machine is ever a bottleneck.

■ Key Takeaway

Start simple with vertical scaling, but design for horizontal scaling early. Nearly every large-scale system you know of runs on many machines working together, not one giant machine.

Today's practice: Think of a shop you visit often. If it suddenly got 10x more customers, would it work better by hiring a super-fast single cashier, or by opening more counters? Why?

Load Balancing — Sharing the Work Fairly

The traffic police officer of your system

Once you have multiple servers (horizontal scaling), something needs to decide which server handles each incoming request. That's the job of a **load balancer** — it sits in front of your servers, receives every request first, and forwards it to whichever server is best placed to handle it.

Common Load Balancing Strategies

- **Round Robin** — requests are handed out to servers one after another, in a repeating cycle.
- **Least Connections** — the request goes to whichever server is currently doing the least work.
- **IP Hash** — a user is consistently sent to the same server based on their location/IP, useful when a server holds session data.
- **Health Checks** — the load balancer regularly pings each server; if one stops responding, it's removed from rotation automatically.

■ Real-Life Analogy

Picture a bank with five teller counters and one traffic officer standing at the entrance. Every customer who walks in is directed by the officer to whichever counter is free or has the shortest queue. Customers never need to guess which counter to go to, and if a teller steps away for a break, the officer simply stops sending people to that counter.

■ Real-World Example

Netflix serves tens of millions of concurrent video streams. A request from your TV first hits a load balancer, which quickly routes it to one of many healthy backend servers — so even during a huge new show's release night, no single server gets overwhelmed while others sit idle.

■ Key Takeaway

A load balancer turns "many servers" into what feels like "one reliable service" from the user's point of view. It's the key piece that makes horizontal scaling actually work in practice.

Today's practice: Next time you're in a queue (bank, airport, supermarket), notice how the queue is organized. Is it round robin, or does everyone join the shortest line? Map that to load balancing strategies.

Caching — Remembering So You Don't Repeat Work

The single most powerful trick for making systems feel instant

A **cache** is a small, fast storage layer that keeps a copy of data that was recently or frequently requested, so the next request for the same data can be answered instantly — without redoing expensive work like a database query or a complex calculation.

■ Real-Life Analogy

Imagine you keep a bottle of water on your desk instead of walking to the kitchen every time you're thirsty. The kitchen is your database — reliable, but a bit of a walk. The bottle on your desk is your cache — instantly available, refilled only when it runs low. You save a huge number of trips by keeping the frequently needed thing close by.

Where Caching Happens

- **Browser cache** — your phone/browser stores images, CSS, and scripts so the app loads faster on your next visit.
- **CDN cache** — servers positioned around the world hold copies of static content close to users (covered on Day 9).
- **Application cache** — tools like Redis or Memcached store frequently-used data (like a user's profile) in memory, which is far faster than reading from a database on disk.
- **Database cache** — the database itself keeps recently used data in memory to avoid slow disk reads.

The Hard Part: Cache Invalidation

The tricky part of caching is knowing when the cached copy has gone *stale* — i.e., the real data has changed but the cache still shows the old value. This is why "cache invalidation" is jokingly called one of the two hardest problems in computer science.

■ Real-World Example

Twitter/X cannot re-calculate your entire home timeline from scratch every single time you refresh the app — that would be far too slow with millions of users refreshing every second. Instead, a pre-built version of your timeline is cached and served instantly, then updated in the background as new posts come in.

■ Key Takeaway

Caching trades a small amount of "freshness" for a massive amount of speed. Use it for data that is read often but doesn't change every second — and always have a plan for how the cache gets updated.

Today's practice: Think of something you do repeatedly that you could instead "cache" in real life — e.g., pre-cooking meals for the week instead of cooking daily. Notice the trade-off between freshness and effort.

Databases — SQL vs NoSQL

Where all your app's data actually lives

A database is where an application permanently stores its data — users, posts, orders, messages. There are two broad families of databases, and picking the right one is one of the first big decisions in any system design.

SQL (Relational) Databases

Examples: MySQL, PostgreSQL, Oracle. Data is stored in strict tables with rows and columns, and every row must follow the same structure (a fixed "schema"). Tables can be linked together through relationships, and SQL databases are excellent at guaranteeing accuracy — for example, ensuring money debited from one account is always credited to another.

NoSQL (Non-Relational) Databases

Examples: MongoDB, Cassandra, DynamoDB. Data is stored more flexibly — often as documents (similar to JSON objects) that don't all need the exact same structure. This flexibility, plus the way NoSQL databases are built, makes them easier to scale horizontally across many machines for very large, fast-growing datasets.

■ Real-Life Analogy

An SQL database is like a strict library catalog system: every book must be entered with a title, author, ISBN, and shelf number in an exact format, and everything is cross-referenced precisely — nothing is ever ambiguous. A NoSQL database is more like a stack of sticky notes on a board: fast to add, flexible in format (one note might have a phone number, another might not), and great when you need speed and flexibility over strict structure.

■ Real-World Example

A bank uses an SQL database because every transaction must be 100% accurate and consistent — no room for error. Instagram uses NoSQL-style databases for things like storing billions of comments and likes, where flexibility and the ability to scale across many servers matters more than rigid structure.

■ Key Takeaway

Choose SQL when your data has clear relationships and accuracy is critical (banking, orders, inventory). Choose NoSQL when you have huge, fast-growing, flexible data and need to scale horizontally with ease (social feeds, logs, real-time data).

Today's practice: Think about the contacts app on your phone (structured, SQL-like) versus a note-taking app where every note can look completely different (flexible, NoSQL-like). Why does each

app choose its structure?

Scaling Databases — Replication & Sharding

What happens when even the database needs more than one machine

A single database server can also become a bottleneck once an app grows large. Two techniques help a database scale: **replication** (copying data) and **sharding** (splitting data).

Replication

Replication means keeping identical copies of the database on multiple servers. Usually there is one "primary" server that handles writes (new/changed data), and several "replica" servers that handle reads (data lookups). This spreads out read traffic and also protects against data loss if one server fails.

Sharding

Sharding means splitting the data itself into smaller pieces ("shards"), each stored on a different server — for example, users A-M on one server and users N-Z on another. This lets a database hold and process far more data than a single machine ever could alone.

■ Real-Life Analogy

Replication is like photocopying a popular reference book and placing copies in five branches of a library, so readers in each branch can consult it without everyone crowding into one building. Sharding is different — it's like splitting a giant encyclopedia into volumes (A-M in one branch, N-Z in another), so no single branch has to store or search through the entire set.

■ Real-World Example

WhatsApp handles billions of messages a day by sharding its data — a user's messages are routed to a specific shard based on their user ID, so no single database server ever has to hold every message from every user in the world.

■ Key Takeaway

Use replication to handle lots of read traffic and to protect against server failure. Use sharding when the data itself is simply too large for one server to store or search through efficiently.

Today's practice: If you managed a library that outgrew one building, would you first try opening branches with copies of the popular books (replication), or splitting the collection across buildings by subject (sharding)? Why might you eventually need both?

CAP Theorem — Picking Your Trade-offs

Why no system can be perfect at everything, all the time

The CAP theorem says that when a database is spread across multiple machines, and a network problem occurs between them (called a "partition"), you can only guarantee two out of these three things at the same time:

- **Consistency (C)** — every server shows the exact same, most up-to-date data at all times.
- **Availability (A)** — every request gets a response, even if it might not be the very latest data.
- **Partition Tolerance (P)** — the system keeps working even if servers can't talk to each other temporarily.

In real-world distributed systems, network partitions *will* happen sooner or later, so partition tolerance is basically mandatory. The real choice most systems face is between consistency and availability when a partition occurs.

■ Real-Life Analogy

Imagine a bank with two branches, and the phone line between them suddenly goes dead. If you walk into Branch A and ask to withdraw money, the branch has two choices: refuse to serve you until it can confirm your balance with Branch B (choosing **consistency** over availability), or serve you immediately using its last known balance, risking a mismatch that gets reconciled later (choosing **availability** over strict consistency).

■ Real-World Example

Traditional banking systems lean towards consistency (CP) — they would rather briefly deny a transaction than risk showing the wrong balance. Amazon's shopping cart and DynamoDB famously lean towards availability (AP) — it's more important that the "Add to Cart" button always works than that every server agrees on the cart's exact contents within the same millisecond.

■ Key Takeaway

There is no single "correct" choice — it depends on what matters more for your specific application: always-correct data, or always-available service. Good system design means consciously choosing this trade-off, not stumbling into it.

Today's practice: Think of an app where you'd rather it be slightly outdated but always working (e.g., a like counter), versus one where you'd rather it refuse to respond than show wrong data (e.g., your bank balance).

Message Queues & Asynchronous Processing

Not everything needs to happen right this second

Some tasks don't need to be completed before responding to the user — like sending a confirmation email after signup, generating a video thumbnail, or notifying a driver of a new ride request. A **message queue** lets one part of a system quickly hand off a task to be done "later," instead of making the user wait for it to finish.

How It Works

A **producer** (e.g., the signup service) drops a message onto a queue (e.g., "send welcome email to user 123") and immediately moves on, responding to the user right away. A separate **consumer** service picks up messages from the queue whenever it's ready and processes them, one by one, without the user ever having to wait. Popular tools for this include Kafka, RabbitMQ, and Amazon SQS.

■ Real-Life Analogy

Think of a restaurant kitchen. When a waiter takes your order, they don't stand at the kitchen window watching your food cook — they clip the order ticket onto a rail and immediately go serve the next table. The kitchen staff (consumers) process tickets from the rail (the queue) in order, at their own pace, while waiters keep taking new orders without ever blocking on any single one.

■ Real-World Example

When you request a ride on Uber, the app instantly confirms your request, while behind the scenes a message queue handles matching you with nearby drivers, calculating fare estimates, and sending notifications — all without making you stare at a frozen loading screen.

■ Key Takeaway

Message queues decouple "doing the work" from "responding to the user," which makes apps feel faster and makes the system more resilient — if the email service is briefly down, messages simply wait in the queue instead of the whole signup process failing.

Today's practice: List three things an app does after you tap a button that probably don't need to finish immediately (e.g., sending a receipt, updating analytics, generating a thumbnail).

CDNs & Microservices — Delivering at Global Scale

Getting content closer to users, and splitting big systems into small ones

Content Delivery Networks (CDNs)

A CDN is a network of servers spread out geographically around the world, each holding cached copies of static content (images, videos, style files). When a user requests that content, it's served from the nearest CDN server instead of traveling all the way back to the original server, which can be on another continent.

■ Real-Life Analogy

Instead of every customer in every city ordering directly from one central factory in another country (slow, expensive shipping), a company opens local warehouses (CDN servers) in each region stocked with popular items. A customer in your city gets their order from the nearby warehouse in hours, not weeks.

Monoliths vs Microservices

A **monolith** is one single, large application where all features (payments, user profiles, search, notifications) live in one codebase and are deployed together. A **microservices** architecture instead splits these features into small, independent services that each do one thing well and can be built, deployed, and scaled separately.

■ Real-Life Analogy

A monolith is like one giant kitchen that cooks every dish on the menu — starters, mains, desserts — all in one space with shared staff and equipment. If the dessert station catches fire, the whole kitchen might have to shut down. Microservices are like a food court with separate stalls: the pizza stall, the noodle stall, and the juice stall each run independently. If the juice stall runs out of oranges, the pizza stall keeps serving customers just fine.

■ Real-World Example

YouTube uses CDNs so that a viral video loads instantly for viewers around the world, not just those near YouTube's main data centers. Netflix runs on microservices — its recommendation engine, billing, and video streaming are all separate services, so a slowdown in recommendations never takes down video playback.

■ Key Takeaway

CDNs bring content physically closer to users to cut down travel time. Microservices break a large system into smaller, independently-scalable pieces so failures and bottlenecks stay isolated instead of taking down the entire application.

Today's practice: Notice how fast images load on a global website versus a small local one. Also, think of a chain restaurant vs a food court — which better resembles microservices, and why?

Putting It All Together — Designing a Real System

A step-by-step framework, applied to a URL shortener like bit.ly

Today, instead of a new concept, you'll combine everything from Days 1-9 into one practical framework that you can reuse for any system design problem — including in interviews.

The 5-Step Framework

- **1. Clarify Requirements** — what must the system do? How many users? How much traffic?
- **2. Estimate Capacity** — roughly how much storage, and how many requests per second, will this need?
- **3. High-Level Design** — sketch the major building blocks: client, load balancer, servers, cache, database.
- **4. Deep Dive** — zoom into the trickiest part (e.g., how exactly do we generate a unique short code?).
- **5. Identify Bottlenecks** — where could this system break under heavy load, and how would you fix it?

Worked Example: Designing a URL Shortener (like bit.ly)

1. Requirements: Users submit a long URL and get back a short one; visiting the short URL redirects to the original. Expect millions of new links per day and far more redirect visits.

2. Capacity: Even at a huge scale of 100 million new links a day, each link is tiny (a few hundred bytes), so storage needs are manageable — the real challenge is handling a very high number of redirect requests per second.

3. High-Level Design: Client sends a long URL to a load balancer, which forwards it to an application server. The server generates a short, unique code, stores the mapping (short code → long URL) in a database, and returns the short link. When someone visits the short link, the server looks up the mapping and redirects them.

4. Deep Dive: To avoid short-code collisions at scale, generate a unique numeric ID for each link (e.g., using a distributed counter) and convert it into a short string of letters and numbers. Since redirects (reads) vastly outnumber link creations (writes), this is a perfect place to add **caching** from Day 4 — store the most popular short codes in a fast in-memory cache so most redirects never even touch the database.

5. Bottlenecks & Fixes: A single database could become a bottleneck at huge scale — apply **replication** from Day 6 to spread out read traffic, and **sharding** if the dataset grows too large for one server. Use a **load balancer** from Day 3 across many application servers, and consider a **CDN** from Day 9 if you're also serving any static assets.

■ Key Takeaway

Every system design problem — no matter how different it looks on the surface — is really about combining the same core building blocks you've now learned: client-server basics, scaling, load balancing, caching, databases, replication/sharding, consistency trade-offs, async processing, and content delivery.

Where to Go From Here

- Practice designing other systems using the same 5-step framework: a chat app, an Instagram feed, a parking system, a ride-sharing app.
- Draw your designs by hand or on a whiteboard tool — visualizing the blocks helps far more than reading alone.
- Read real engineering blogs from companies like Netflix, Uber, and Meta to see these concepts applied at real scale.
- Revisit Days 1-9 after finishing a few practice designs — the concepts will make even more sense the second time.

Congratulations on completing the 10-day roadmap. You now have the vocabulary and mental models that most beginners take months to build. The next step is simply practice — design something new every few days, and the pieces will keep clicking into place.